

Universality of consensus

6

6.1 Introduction

In Chapter 5, we considered a simple technique for proving statements of the form “there is no wait-free implementation of X by Y .” We considered object classes with deterministic sequential specifications.¹ We derived a hierarchy in which no object from one level can implement an object at a higher level (see Fig. 6.1). Recall that each object has an associated *consensus number*, which is the maximum number of threads for which the object can solve the consensus problem. In a system of n or more concurrent threads, it is impossible to construct a wait-free implementation of an object with consensus number n from objects with lower consensus numbers. The same result holds for lock-free implementations, and henceforth unless we explicitly state otherwise, it is implied that a result that holds for wait-free implementations holds for lock-free ones as well.

The impossibility results of Chapter 5 do not by any means imply that wait-free synchronization is impossible or infeasible. In this chapter, we show that there are classes of objects that are *universal*: Given sufficiently many of them, one can construct a wait-free linearizable implementation of *any* concurrent object.

A class is universal in a system of n threads if and only if it has a consensus number greater than or equal to n . In Fig. 6.1, each class at level n is universal for a system of n threads. A machine architecture or programming language is computationally powerful enough to support arbitrary wait-free synchronization if and only if it provides objects of a universal class as primitives. For example, modern multiprocessor machines that provide a `compareAndSet()` operation are universal for any number of threads: They can implement any concurrent object in a wait-free manner.

This chapter describes a *universal construction* that implements any concurrent object from consensus objects. The chapter does *not* describe practical techniques for implementing wait-free objects. Like classical computability theory, understanding the universal construction and its implications allows us to avoid the naïve mistake of trying to solve unsolvable problems. Once we understand *why* consensus is powerful enough to implement any kind of object, we will be better prepared to undertake the engineering effort needed to make such constructions efficient.

¹ The situation with nondeterministic objects is significantly more complicated.

Consensus	
Number	Object
1	atomic registers
2	getAndSet(), getAndAdd(), Queue, Stack
⋮	⋮
m	$(m, m(m+1)/2)$ -assignment
⋮	⋮
∞	memory-to-memory move, compareAndSet(), load-linked/store-conditional ^a

^a See Appendix B for details.

FIGURE 6.1

Concurrent computability and the universality hierarchy of synchronization operations.

6.2 Universality

A class C is *universal* if one can construct a wait-free implementation of any object from some number of objects of C and some number of read–write registers. Our construction uses multiple objects of class C because we are ultimately interested in understanding the synchronization power of machine instructions, and most machines allow their instructions to be applied to multiple memory locations. We allow an implementation to use multiple read–write registers because it is convenient for bookkeeping, and memory is usually in plentiful supply on modern architectures. To avoid distraction, we use an unlimited number of read–write registers and consensus objects, leaving the question of recycling memory as an exercise. We begin by presenting a lock-free implementation, later extending it to a slightly more complex wait-free one.

6.3 A lock-free universal construction

Fig. 6.2 shows a *generic* definition for a sequential object, based on the invocation–response formulation of Chapter 3. Each object is created in a fixed initial state. The `apply()` method takes as argument an *invocation* which describes the method being called and its arguments, and returns a *response* containing the call’s termination

```
1 public interface SeqObject {  
2     public abstract Response apply(Invoc invoc);  
3 }
```

FIGURE 6.2

A generic sequential object: The `apply()` method applies the invocation and returns a response.

```

1 public class Node {
2     public Invoc invoc;           // method name and args
3     public Consensus<Node> decideNext; // decide next Node in list
4     public Node next;             // the next node
5     public int seq;               // sequence number
6     public Node(Invoc invoc) {
7         invoc = invoc;
8         decideNext = new Consensus<Node>()
9         seq = 0;
10    }
11    public static Node max(Node[] array) {
12        Node max = array[0];
13        for (int i = 1; i < array.length; i++)
14            if (max.seq < array[i].seq)
15                max = array[i];
16        return max;
17    }
18 }

```

FIGURE 6.3

The Node class.

condition (normal or exceptional) and the return value, if any. For example, a stack invocation might be `push()` with an argument, and the corresponding response would be normal and **void**.

Figs. 6.3 and 6.4 show a universal construction that transforms any sequential object into a lock-free linearizable concurrent object. This construction assumes that sequential objects are *deterministic*: If we apply a method to an object in a particular state, then there is only one possible response and one possible new object state. We can represent any object as a combination of a sequential object in its initial state and a *log*: a linked list of nodes representing the sequence of method calls applied to the object (and hence the object's sequence of state transitions). A thread executes a method call by adding the new call to the head of the list. It then traverses the list, from tail to head, applying the method calls to a private copy of the object. The thread finally returns the result of applying its own operation. It is important to understand that only the head of the log is mutable: The initial state and nodes preceding the head never change.

How do we make this log-based construction concurrent, that is, allow threads to make concurrent calls to `apply()`? A thread attempting to call `apply()` creates a node to hold its invocation. The threads then compete to append their respective nodes to the head of the log by running an n -thread consensus protocol to agree which node was appended to the log. The inputs to this consensus are references to the threads' nodes, and the result is the unique winning node.

The winner can then proceed to compute its response. It does so by creating a local copy of the sequential object and traversing the log, following next references

```

1  public class LfUniversal {
2      private Node[] head;
3      private Node tail;
4      public LfUniversal() {
5          tail = new Node();
6          tail.seq = 1;
7          for (int i = 0; i < n; i++)
8              head[i] = tail
9      }
10     public Response apply(Invoc invoc) {
11         int i = ThreadID.get();
12         Node prefer = new Node(invoc);
13         while (prefer.seq == 0) {
14             Node before = Node.max(head);
15             Node after = before.decideNext.decide(prefer);
16             before.next = after;
17             after.seq = before.seq + 1;
18             head[i] = after;
19         }
20         SeqObject myObject = new SeqObject();
21         Node current = tail.next;
22         while (current != prefer){
23             myObject.apply(current.invoc);
24             current = current.next;
25         }
26         return myObject.apply(current.invoc);
27     }
28 }

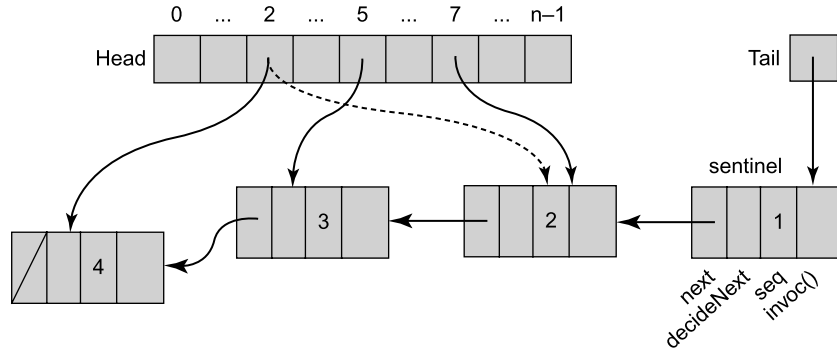
```

FIGURE 6.4

The lock-free universal construction.

from tail to head, applying the operations in the log to its copy, finally returning the response associated with its own invocation. This algorithm works even when `apply()` calls are concurrent because the prefix of the log up to the thread's own node never changes. The losing threads, which were not chosen by the consensus object, must try again to set the node currently at the head of the log (which changes between attempts) to point to them.

We now consider this construction in detail. The code for the lock-free universal construction appears in Fig. 6.4. A sample execution appears in Fig. 6.5. The object state is defined by a linked list of nodes, each one containing an invocation. The code for a node appears in Fig. 6.3. The node's `decideNext` field is a consensus object used to decide which node is appended next in the list, and `next` is the field in which the outcome of that consensus, the reference to the next node, is recorded. The `seq` field is the node's sequence number in the list. This field is 0 while the node is not

**FIGURE 6.5**

Execution of the lock-free universal construction. Thread 2 appends the second node in the log winning consensus on `decideNext` in the sentinel node. It then sets the node's sequence number from 0 to 2, and refers to it from its entry in the `head[]` array. Thread 7 loses the `decideNext` consensus at the sentinel node, sets the `next` reference and sequence number of the decided successor node to 2 (they were already set to the same values by thread 2), and refers to the node from its entry in the `head[]` array. Thread 5 appends the third node, updates its sequence number to 3, and updates its entry in the `head[]` array to this node. Finally, thread 2 appends the fourth node, sets its sequence number to 4, and refers to it from its entry in the `head[]` array. The maximal value in the head array keeps track of the head of the log.

yet threaded onto the list, and positive otherwise. Sequence numbers for successive nodes in the list increase by 1. Initially, the log consists of a unique sentinel node with sequence number 1.

The hard part about designing the concurrent lock-free universal construction is that consensus objects can be used only once.²

In our lock-free algorithm in Fig. 6.4, each thread allocates a node holding its invocation, and repeatedly tries to append that node to the head of the log. Each node has a `decideNext` field, which is a consensus object. A thread tries to append its node by proposing it as input to a consensus protocol on the head's `decideNext` field. Because threads that do not participate in this consensus may need to traverse the list, the result of this consensus is stored in the node's `next` field. Multiple threads may update this field simultaneously, but they all write the same value. When a thread appends a node, it sets the node's sequence number.

Once a thread's node is part of the log, the thread computes the response to its invocation by traversing the log from the tail to the newly added node. It applies each of the invocations to a private copy of the object, and returns the response from its

² Creating a reusable consensus object, or even one whose decision is readable, is not a simple task. It is essentially the same problem as the universal construction we are about to design. For example, consider the queue-based consensus protocol in Section 5.4. It is not obvious how to use a Queue to allow repeated reading of the consensus object state after it is decided.

own invocation. Note that when a thread computes its response, all its predecessors' next references must already be set, because these nodes have already been added to the head of the list. Any thread that added a node to the list has updated the next reference of its predecessor with the result of the `decideNext` consensus.

How do we locate the head of the log? We cannot track the head with a consensus object because the head must be updated repeatedly, and consensus objects can only be accessed once by each thread. Instead, we create a per-thread structure of the kind used in the bakery algorithm (Section 2.7). We use an n -entry array `head[]`, where `head[i]` is the last node in the list that thread i has observed. Initially all entries refer to the `tail` sentinel node. The head is the node with the maximum sequence number among the nodes referenced in the `head[]` array. The `max()` method in Fig. 6.3 performs a collect, reading `head[]` and returning the node with the highest sequence number.

The construction is a linearizable implementation of the sequential object. Each `apply()` call can be linearized to the `decide()` call adding the node to the log.

Why is this construction lock-free? The head of the log, the latest node appended, is added to the `head[]` array within a finite number of steps. The node's predecessor must appear in the head array, so any node repeatedly attempting to add a new node will repeatedly run the `max()` function on the head array. It detects this predecessor, applies consensus on its `decideNext` field, and then updates the winning node's fields, including its sequence number. Finally, it stores the decided node in that thread's head array entry. The new head node always eventually appears in `head[]`. It follows that the only way a thread can repeatedly fail to add its own node to the log is if other threads repeatedly succeed in appending their own nodes to the log. Thus, a node can starve only if other nodes are continually completing their invocations, implying that the construction is lock-free.

6.4 A wait-free universal construction

How do we make a lock-free algorithm wait-free? The full wait-free algorithm appears in Fig. 6.6. We must guarantee that every thread completes an `apply()` call within a finite number of steps; that is, no thread starves. To guarantee this property, threads making progress help less fortunate threads complete their calls. This *helping* pattern shows up later in a specialized form in other wait-free algorithms.

To enable helping, each thread shares with other threads the `apply()` call that it is trying to complete. We add an n -element `announce[]` array, where `announce[i]` is the node that thread i is currently trying to append to the list. Initially, all entries refer to the sentinel node, which has a sequence number 1. Thread i *announces* a node when it stores the node in `announce[i]`.

To execute `apply()`, a thread first announces its new node. This step ensures that if the thread itself does not succeed in appending its node onto the list, some other thread can append that node on its behalf. It then proceeds as before, attempting to append the node into the log. To do so, it reads the `head[]` array only once (line 15), and then enters the main loop of the algorithm, which it executes until its own node

```

1  public class Universal {
2      private Node[] announce; // array added to coordinate helping
3      private Node[] head;
4      private Node tail = new Node();
5      public Universal() {
6          tail.seq = 1;
7          for (int j = 0; j < n; j++) {
8              head[j] = tail;
9              announce[j] = tail;
10         }
11     }
12     public Response apply(Invoc invoc) {
13         int i = ThreadID.get();
14         announce[i] = new Node(invoc);
15         head[i] = Node.max(head);
16         while (announce[i].seq == 0) {
17             Node before = head[i];
18             Node help = announce[(before.seq + 1) % n];
19             if (help.seq == 0)
20                 prefer = help;
21             else
22                 prefer = announce[i];
23             Node after = before.decideNext.decide(prefer);
24             before.next = after;
25             after.seq = before.seq + 1;
26             head[i] = after;
27         }
28         head[i] = announce[i];
29         SeqObject myObject = new SeqObject();
30         Node current = tail.next;
31         while (current != announce[i]){
32             myObject.apply(current.invoc);
33             current = current.next;
34         }
35         return myObject.apply(current.invoc);
36     }
37 }

```

FIGURE 6.6

The wait-free universal construction.

has been appended to the list (detected on line 16 after its sequence number becomes nonzero). Here is a change from the lock-free algorithm. A thread first checks to see if there is a node that needs help ahead of it in the `announce[]` array (line 18). The node to be helped must be determined dynamically because nodes are continually added to the log. A thread attempts to help nodes in the `announce[]` array in increasing order,

determined by the sequence number modulo the width n of the announce[] array. We prove that this approach guarantees that any node that does not make progress on its own will eventually be helped by others once its owner thread's index matches the maximal sequence number modulo n . If this helping step were omitted, then an individual thread could be overtaken an arbitrary number of times. If the node selected for help does not require help (i.e., its sequence number is nonzero in line 19), then each thread attempts to append its own node (line 22). (All announce[] array entries are initialized to the sentinel node, with sequence number 1.) The rest of the algorithm is almost the same as in the lock-free algorithm. A node is appended when its sequence number becomes nonzero. In this case, the thread proceeds as before to compute its result based on the immutable segment of the log from the tail to its own node.

Fig. 6.7 shows an execution of the wait-free universal construction in which, starting from the initial state, thread 5 announces its new node and appends it to the

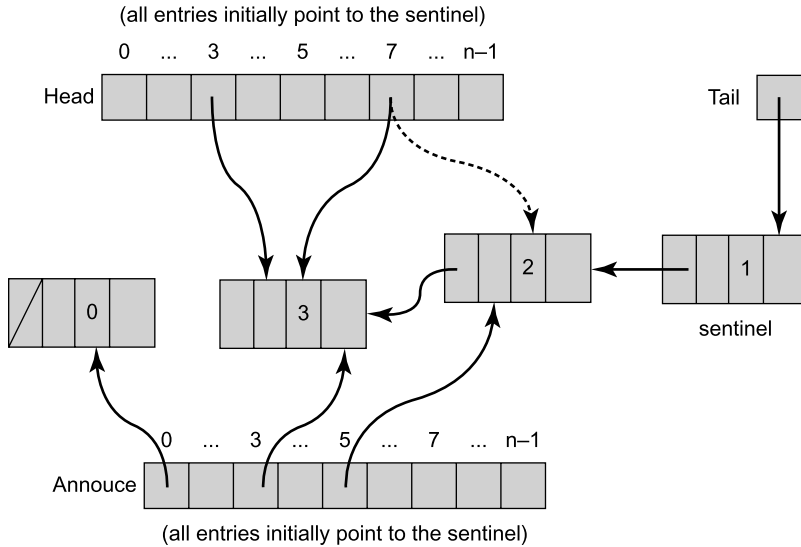


FIGURE 6.7

Execution of the wait-free universal construction. Thread 5 announces its new node and appends it to the log, but halts before adding it to the head[] array. Thread 7 does not see thread 5's node in the head[] array. Since thread 2 (whose ID is $(\text{before.seq} + 1) \bmod n$) is not trying to add a node, thread 7 tries to add its own node. However, it loses the consensus on the sentinel node's `decideNext` object since thread 5 already won. Thread 7 therefore completes updating the fields of thread 5's node, setting the node's sequence number to 2, and adding the node to the head[] array. Note that thread 5's own entry in the head[] array is not yet set to its announced node. Next, thread 3 announces its node and then pauses before entering the main loop. Thread 7 now successfully helps thread 3, appending its node and setting its sequence number to 3. Now thread 3 wakes up. It does not enter the main loop because its node's sequence number is nonzero, but will update the head[] array and compute its output value using a copy of the sequential object.

log, and pauses before adding the node to `head[]`. Thread 7 then takes steps. The value of $(\text{before.seq} + 1) \bmod n$ is 2, but thread 2 is not trying to add a node, so thread 7 tries to add its own node. It loses the consensus on the sentinel node's `decideNext` object since thread 5 already won, and thus completes the operation of thread 5, setting the node's sequence number to 2 and adding the node to the `head[]` array. Next, thread 3 announces its node and pauses before entering the main loop. Then thread 7 helps thread 3: it appends thread 3's node, but pauses after setting its sequence number to 3 but before adding the node to `head[]`. Now thread 3 wakes up. It does not enter the main loop because its node's sequence number is nonzero, but updates `head[]` on line 28 and computes its output value using a copy of the sequential object.

There is a delicate point to understand about these modifications to the lock-free algorithm. Since more than one thread can attempt to append a particular node to the log, we must make sure that no node is appended twice. One thread might append the node and set the node's sequence number at the same time that another thread appends the same node and sets its sequence number. The algorithm avoids this error because of the order in which threads read the maximum `head[]` array value and the sequence number of a node in the `announce[]` array. Let a be a node created by thread A and appended by threads A and B . It must be added at least once to `head[]` before the second append. Note, however, that the before node read from `head[A]` by B (line 17) must be a itself, or a successor of a in the log. Moreover, before any node is added to `head[]` (either on line 26 or on line 28), its sequence number is made nonzero (line 25). The order of operations ensures that B sets its `head[B]` entry (the entry based on which B 's `before` variable will be set, resulting in an erroneous append) in line 15 or line 26, and only then validates that the sequence number of a is nonzero in line 16 or line 19 (depending on whether A or another thread performs the operation). It follows that the validation of the erroneous second append will fail because the sequence number of node a will already be nonzero, and it will not be added to the log a second time.

Linearizability follows because no node is ever added twice, and the order in which nodes are appended to the log is clearly compatible with the natural partial order of the corresponding method calls.

To prove that the algorithm is wait-free, we must show that the helping mechanism guarantees that any node that is announced is eventually added to the `head[]` array (implying that it is in the log) and the announcing thread can complete computation of its outcome. To assist in the proof, it is convenient to define some notation. Let $\text{max}(\text{head}[])$ be the node with the largest sequence number in the `head[]` array, and let " $c \in \text{head}[]$ " denote the assertion that `head[i]` is set to node c , for some i .

An *auxiliary* variable (sometimes called a *ghost* variable) is one that does not appear explicitly in the code, does not alter the program's behavior in any way, and yet helps us reason about the behavior of the algorithm. We use the following auxiliary variables:

- $\text{concur}(A)$ is the set of nodes that have been stored in the `head[]` array since thread A 's last announcement.
- $\text{start}(A)$ is the sequence number of $\text{max}(\text{head}[])$ when thread A last announced.

```

12 public Response apply(Invoc invoc) {
13     int i = ThreadID.get();
14     < announce[i] = new Node(invoc); start(i) = max(head); concur(i) = {}; >
15     head[i] = Node.max(head);
16     while (announce[i].seq == 0) {
17         Node before = head[i];
18         Node help = announce[(before.seq + 1) % n];
19         if (help.seq == 0)
20             prefer = help;
21         else
22             prefer = announce[i];
23         Node after = before.decideNext.decide(prefer);
24         before.next = after;
25         after.seq = before.seq + 1;
26         < head[i] = after; (∀j) (concur(j) = concur(j) ∪ {after}); >
27     }
28     < head[i] = announce[i]; (∀j) (concur(j) = concur(j) ∪ {after}); >
29     SeqObject MyObject = new SeqObject();
30     Node current = tail.next;
31     while (current != announce[i]){
32         MyObject.apply(current.invoc);
33         current = current.next;
34     }
35     return MyObject.apply(current.invoc);
36 }

```

FIGURE 6.8

The `apply()` method of the wait-free universal construction with auxiliary variables. Operations in angled brackets are assumed to happen atomically.

The code reflecting the auxiliary variables and how they are updated appears in Fig. 6.8. For example, the statement

$$(\forall j) (\text{concur}(j) = \text{concur}(j) \cup \{\text{after}\});$$

means that the node *after* is added to *concur(j)* for all threads *j*. The code statements within the angled brackets are considered to be executed atomically. This atomicity can be assumed because auxiliary variables do not affect the computation in any way. For brevity, we slightly abuse the notation by letting the function `max()` applied to a node or array of nodes return the maximal among their sequence numbers.

Note the following property is invariant throughout the execution of the universal construction:

$$|\text{concur}(A)| + \text{start}(A) = \max(\text{head}[]). \quad (6.4.1)$$

Lemma 6.4.1. For all threads *A*, the following claim is always true:

$$|\text{concur}(A)| > n \Rightarrow \text{announce}[A] \in \text{head}[].$$

Proof. Let $a = \text{announce}[A]$. If $|\text{concur}(A)| > n$, then $\text{concur}(A)$ includes successive nodes b and c (appended to the log by threads B and C) whose respective sequence numbers plus 1 are equal to $A - 1$ and A modulo n (note that B and C are the threads that add b and c to the log, not necessarily the ones that announced them). Thread C appends to the log the node located in $\text{announce}[A]$ at the time it executes lines 18–22, unless it had already been added to the log. We need to show that when C reads $\text{announce}[A]$, A has already announced a , so c adds a to the log, or a was already added. Later, when c is added to $\text{head}[]$ and $|\text{concur}(A)| > n$, a will be in $\text{head}[]$ as the lemma requires.

To see why a must have already been announced when C reached lines 18–22, note that (1) because C appended its node c to b , it must have read b as the *before* node on line 17, implying that B added b to the log before it was read from $\text{head}[]$ by C on line 17, and (2) because b is in $\text{concur}(A)$, A announced a before b was added to $\text{head}[]$. From (1) and (2), it follows that A announced before C executed lines 18–22, and the claim follows. $\square$

Lemma 6.4.1 places a bound on the number of nodes that can be appended while a method call is in progress. We now give a sequence of lemmas showing that when A finishes scanning the $\text{head}[]$ array, either $\text{announce}[A]$ is appended or $\text{head}[A]$ lies within $n + 1$ nodes of the end of the list.

Lemma 6.4.2. The following property always holds:

$$\max(\text{head}[]) \geq \text{start}(A).$$

Proof. The sequence number for each $\text{head}[i]$ is nondecreasing. $\square$

Lemma 6.4.3. The following is a loop invariant for line 13 of Fig. 6.3 (i.e., it holds during each iteration of the loop):

$$\max(\max, \text{head}[i], \dots, \text{head}[n - 1]) \geq \text{start}(A),$$

where i is the loop index, $\max$ is the node with the maximum sequence number found so far, and A is the thread executing the loop.

In other words, the maximum sequence number of $\max$ and all $\text{head}[]$ entries from the current value of i to the end of the loop never become smaller than the maximum value in the array when A announced.

Proof. When i is 1, the assertion is implied by Lemma 6.4.2 (since $\max = \text{head}[0]$). The truth of the assertion is preserved at each iteration, when $\max$ is replaced by the node with the sequence number $\max(\max, \text{head}[i])$. $\square$

Lemma 6.4.4. The following assertion holds just before line 16 (of Fig. 6.8):

$$\text{head}[A].\text{seq} \geq \text{start}(A).$$

Proof. After the call to `Node.max()` at line 15, the result follows from Lemma 6.4.3. Otherwise, `head[A]` is set to point to A 's last appended node on line 26, which increases `head[A].seq` by 1. $\square$

Lemma 6.4.5. The following property always holds:

$$|\text{concur}(A)| \geq \text{head}[A].\text{seq} - \text{start}(A) \geq 0.$$

Proof. The lower bound follows from Lemma 6.4.4, and the upper bound follows from Eq. (6.4.1). $\square$

Theorem 6.4.6. The algorithm in Fig. 6.6 is correct and wait-free.

Proof. To see that the algorithm is wait-free, note that A can execute the main loop no more than $n + 1$ times. At each successful iteration, `head[A].seq` increases by 1. After $n + 1$ iterations, Lemma 6.4.5 implies that

$$|\text{concur}(A)| \geq \text{head}[A].\text{seq} - \text{start}(A) \geq n.$$

Lemma 6.4.1 implies that `announce[A]` must have been added to `head[]`. $\square$

6.5 Chapter notes

The universal construction described here is adapted from Maurice Herlihy's 1991 paper [69]. An alternative lock-free universal construction using *load-linked/store-conditional* appears in [65]. The complexity of this construction can be improved in several ways. Yehuda Afek, Dalia Dauber, and Dan Touitou [3] showed how to improve the time complexity to depend on the number of concurrent threads, not the maximum possible number of threads. Mark Moir [130] showed how to design lock-free and wait-free constructions that do not require copying the entire object. James Anderson and Mark Moir [11] extended the construction to allow multiple objects to be updated. Prasad Jayanti [89] showed that any universal construction has worst-case $\Omega(n)$ complexity, where n is the maximal number of threads. Tushar Chandra, Prasad Jayanti, and King Tan [27] identified a large class of objects for which a more efficient universal construction exists.

Our classification of dependent progress conditions has implications for the foundations of shared-memory computability. Lamport's register-based approach to read-write memory computability [99,105] is based on wait-free implementations of one register type from another. Similarly, Herlihy's consensus hierarchy [69] applies to wait-free or lock-free object implementations. Combined, these structures form the basis of a theory of concurrent shared-memory computability that explains what objects can be used to implement other objects in an asynchronous shared-memory multiprocessor environment. One might ask why such a theory should rest on non-blocking progress conditions (that is, wait-free or lock-free) and not on locks. After

all, locking implementations are common in practice. Moreover, the obstruction-free condition is a nonblocking progress condition where read–write registers are universal [68], effectively leveling the consensus hierarchy. We are now in a position to address this question. Perhaps surprisingly, Fig. 3.10 suggests that the lock-free and wait-free conditions provide a sound basis for a concurrent computability theory because they are independent progress conditions (i.e., they do not rely on the good behavior of the operating system scheduler). A theory based on a dependent condition would require strong, perhaps arbitrary assumptions about the environment in which programs are executed. When studying the computational power of synchronization primitives, it is unsatisfactory to rely on the operating system to ensure progress, both because it obscures the inherent synchronization power of the primitives, and because we might want to use such primitives in the construction of the operating system itself. For these reasons, a satisfactory theory of shared-memory computability should rely on independent progress conditions such as wait-freedom or lock-freedom, not on dependent properties.

6.6 Exercises

Exercise 6.1. Consider a concurrent atomic `PeekableStack( $k$ )` object: an atomic Stack with an added `look` operation. It allows each of n threads to execute `push()` and `pop()` operations atomically with the usual LIFO semantics. In addition, it offers a `look` operation, the first k calls of which return the value at the bottom of the stack (the least recently pushed value that is currently in the stack) without popping it. All subsequent calls to `look` after the first k return *null*. Also, `look` returns *null* when the Stack is empty.

- Is it possible to construct a wait-free Queue (accessed by at most two threads) from an arbitrary number of `PeekableStack(1)` (i.e., with $k = 1$) objects and atomic read–write registers? Prove your claim.
- Is it possible to construct a wait-free n -thread `PeekableStack(2)` object from an arbitrary number of atomic Stack objects and atomic read–write registers? Prove your claim.

Exercise 6.2. Give an example showing how the universal construction can fail for objects with nondeterministic sequential specifications.

Exercise 6.3. Propose a way to fix the universal construction of Fig. 6.8 to work for objects with nondeterministic sequential specifications.

Exercise 6.4. In both the lock-free and wait-free universal constructions, the sequence number of the sentinel node at the `tail` of the list is initially set to 1. Which of these algorithms, if any, would cease to work correctly if the sentinel node’s sequence number were initially set to 0?

Exercise 6.5. In the lock-free universal construction, every thread has its own view of the head pointer. To append a new method invocation, at line 14 of Fig. 6.4, a thread selects the furthest among these head pointers:

```
Node before = Node.max(head);
```

Consider changing this line to:

```
Node before = head[i];
```

Does the construction still work?

Exercise 6.6. Suppose, instead of a universal construction, you simply want to use consensus to implement a wait-free linearizable register with `read()` and `compareAndSet()` methods. Show how you would adapt this algorithm to do so.

Exercise 6.7. In the wait-free universal construction shown in Section 6.4, each thread first looks for another thread to help, and then tries to append its own node.

Suppose that instead, each thread first tries to append its own node, and then tries to help the other thread. Explain whether this alternative approach works. Justify your answer.

Exercise 6.8. In the construction in Fig. 6.4, we use a “distributed” implementation of a “head” reference (to the node whose `decideNext` field it will try to modify) to avoid having to create an object that allows repeated consensus. Replace this implementation with one that has no head reference at all, and finds the next “head” by traversing down the log from the start until it reaches a node with a sequence number of 0 or with the highest nonzero sequence number.

Exercise 6.9. In the wait-free protocol, a thread adds its newly appended node to the `head[]` array on line 28 even though it may have already added it on line 26. This is done because, unlike in the lock-free protocol, it could be that the thread’s node was added by another thread on line 26, and that “helping” thread stopped at line 26 right after updating the node’s sequence number but before updating the `head[]` array.

1. Explain how removing line 28 would violate Lemma 6.4.4.
2. Would the algorithm still work correctly?

Exercise 6.10. Propose a way to fix the universal construction to work with a bounded amount of memory, that is, a bounded number of consensus objects and a bounded number of read–write registers.

Hint: Add a `before` field to the nodes and build a memory recycling scheme into the code.

Exercise 6.11. Implement a consensus object that is accessed more than once by each thread using `read()` and `compareAndSet()` methods, creating a “multiple access” consensus object. Do not use the universal construction.

Exercise 6.12. Your mission is to transform a sequential stack implementation into a wait-free, linearizable stack implementation, without regard for questions of efficiency or memory use.

You are given a “black-box” Sequence type with the following methods: You can atomically *append* an item to the end of the sequence. For example, if the sequence is $\langle 1, 2, 3 \rangle$, and you append 4, the sequence becomes $\langle 1, 2, 3, 4 \rangle$. This operation is wait-free and linearizable: if a concurrent thread tries to append 5, the sequence becomes either $\langle 1, 2, 3, 4, 5 \rangle$. or $\langle 1, 2, 3, 5, 4 \rangle$. Note that Sequence items *do not* have to be integers: they can be any kind of object you like.

You can also iterate through the elements of a sequence. Here, we iterate through a sequence printing each value until we see the string “stop”.

```
1 foreach x in s {  
2   if (x == "stop") break;  
3   System.out.println(x)  
4 }
```

(Note that if another thread is appending new values while you are iterating through a sequence, you might keep going forever.)

Implement a wait-free linearizable stack using an atomic sequence object, and as much atomic read–write memory and sequential stack objects as you like. Your stack should support both `push()` and `pop()` operations with the usual meanings. Again, do not worry about efficiency or memory use.

Explain briefly why your construction is wait-free and linearizable (in particular, identify the linearization points).

Spin locks and contention

7

When writing programs for uniprocessors, it is usually safe to ignore the underlying system's architectural details. Unfortunately, multiprocessor programming has yet to reach that state; for now, it is crucial to understand the underlying machine architecture. The goal of this chapter is to explain how architecture affects performance, and how to exploit this knowledge to write efficient concurrent programs. We revisit the familiar mutual exclusion problem, this time with the aim of devising mutual exclusion protocols that work well with today's multiprocessors.

Any mutual exclusion protocol poses the question: "What do you do if you cannot acquire the lock?" There are two alternatives. If you keep trying, the lock is called a *spin lock*, and repeatedly testing the lock is called *spinning*, or *busy-waiting*. The Filter and Bakery algorithms are spin locks. Spinning makes sense when you expect the lock delay to be short (and only on multiprocessors, of course). The alternative is to suspend yourself and ask the operating system to schedule another thread on your processor, which is sometimes called *blocking*. Because switching from one thread to another is expensive, blocking makes sense only if you expect the lock delay to be long. Many operating systems mix both strategies, spinning for a short time and then blocking. Both spinning and blocking are important techniques. In this chapter, we turn our attention to locks that use spinning.

7.1 Welcome to the real world

We approach real-world mutual exclusion via the `Lock` interface from the `java.util.concurrent.locks` package. For now, we consider only the two principal methods, `lock()` and `unlock()`. As mentioned in [Pragma 2.2.1](#), these methods are often used in the following structured way:

```

1 Lock mutex = new LockImpl(...); // lock implementation
2 ...
3 mutex.lock();
4 try {
5     ...           // body
6 } finally {
7     ... // restore object invariant if needed
8     mutex.unlock();
9 }
```